

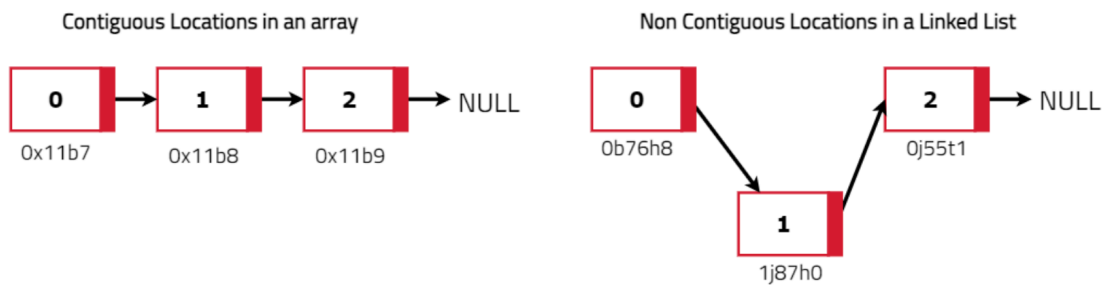
linked list

A linked list is a data structure where each element points to the next, allowing flexible addition and removal of elements.

These collection of elements are called **nodes**, and each **node** represent an individual element

The elements are not stored contiguously in memory, unlike arrays

▼ Where do we need linked lists over arrays?



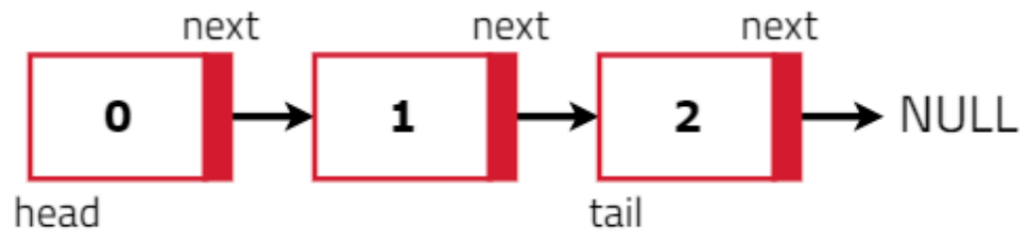
When the size of the collection of elements is dynamic

→ Arrays are of fixed size. Their capacity can neither be increased nor decreased once initialized.

→ Hence, expanding an array beyond it's initial size can be challenging and inefficient

→ However, linked lists are dynamic in nature and can grow and shrink as needed.

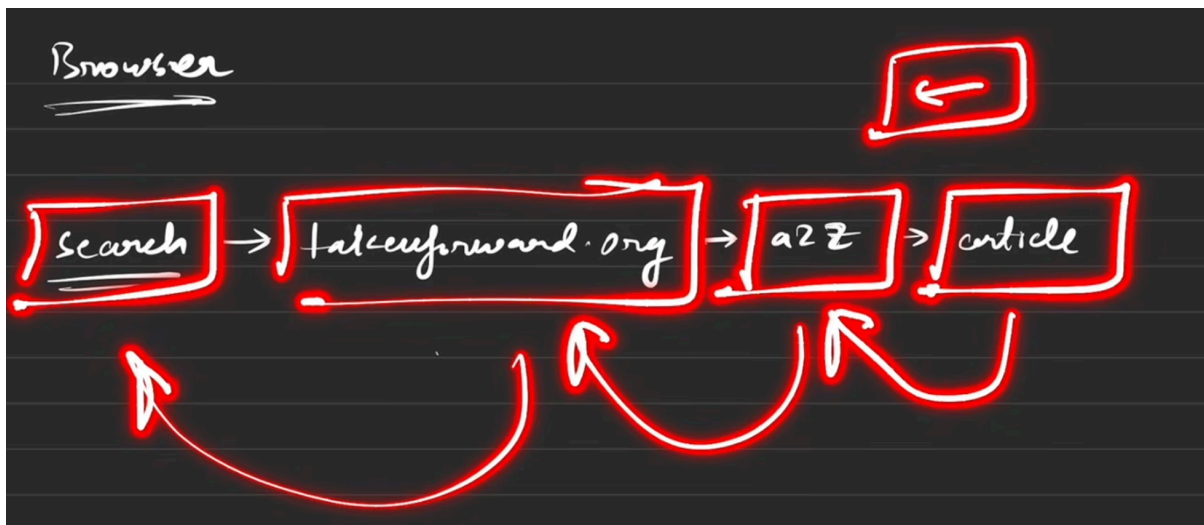
▼ How is a linked list arranged and what does it contain internally?



The linked list data structure contains two crucial pieces of information. The first being the **data** and the other being the pointer/**reference** to the next element

→ The **head** is the first node and **tail** is the last node in a linked list

▼ Where is linked list used in real life?



Linked lists are always used inside browsers

▼ What is a 1D linked list?

Where we only know the location of the next node, and have no information regarding the previous node, hence we are only able to navigate in one

direction

▼ How can we implement a Linked List in Java?

We know that a linked list must contain two components, one is the data and the other is the reference to the next node.

Now, the data type of the node's data should be user defined, hence we should make the Node Generic

▼ Java Code

```
public class GenericNode <T> {  
  
    public T data;  
    public GenericNode<T> next;  
  
    public GenericNode(T data) {  
        this.data = data;  
        next = null;  
    }  
}
```

```
import java.util.Iterator;  
  
public class GenericArray<T> implements Iterable{  
  
    private final T[] items;  
    private int length;  
  
    public GenericArray() {  
        items = (T[]) new Object[100];  
        length = 0;  
    }  
  
    public void add(T data) {  
        items[length++] = data;  
    }  
}
```

```

    }

    public T get(int index) {
        if (index >= length) {
            throw new RuntimeException("Index out of Bounds");
        }
        return items[index];
    }

    public int size() {
        return length;
    }

    @Override
    public Iterator iterator() {
        return new GenericIterator(this);
    }

    private class GenericIterator implements Iterator {

        private final GenericArray<T> list;
        private int index;

        public GenericIterator(GenericArray<T> list) {
            this.list = list;
            index = 0;
        }

        @Override
        public boolean hasNext() {
            return index < list.length;
        }

        @Override
        public T next() {

```

```

        return list.items[index ++];
    }
}

```

```

public class Main {

    private static <T> GenericNode convertListIntoLinkedList(
GenericArray<T> list) {
        GenericNode<T> head = new GenericNode<>(list.get
(0));
        GenericNode<T> mover = head;

        for (int i = 1; i < list.size(); i++) {
            GenericNode<T> child = new GenericNode<>(lis
t.get(i));
            mover.next = child;
            mover = child;
        }

        return head;
    }

    private static <T> void printLinkedList(GenericNode<
T> head) {
        GenericNode<T> mover = head;
        while (mover != null) {
            System.out.print(mover.next != null ? mover.
data + " -> " : mover.data);
            mover = mover.next;
        }
    }

    public static void main(String[] args) {
        GenericArray<Integer> numbers = new GenericArray

```

```
<>();  
    numbers.add(1); numbers.add(2); numbers.add(3);  
    numbers.add(4); numbers.add(5);  
  
    GenericNode<Integer> head = convertListIntoLink  
dList(numbers);  
    printLinkedList(head);  
}  
}
```